



YOUR TECHNOLOGY & INNOVATION PARTNER

Building Multi-Agent Systems

Agenda

- 1. Course Objectives & Learning Outcomes**
- 2. From Single-Agent Systems to Multi-Agent**
- 3. Multi-Agent Architectures & Design Patterns with ADK**
- 4. Agent Communication & Memory Management in ADK**
- 5. Integrating ADK Agents with Tools**
- 6. Skills for ADK Agents**
- 7. Demo**
- 8. Q&A and Technical Discussion**

1. Course Objectives & Learning Outcomes

This section defines the course goals, target audience, and expected outcomes, providing a clear roadmap toward building production-grade multi-agent systems with Google Agent Development Kit.

Course Overview

Goal of This Course

- Understand the **fundamentals of Multi-Agent Systems (MAS)**
- Learn how to **design scalable agent architectures**
- Build **production-ready MAS using Google ADK**
- Integrate **memory, tools, and skills** into agent workflows

Focus

- Practical architecture
- Engineering mindset
- Production-oriented design

Key Competencies After Completion

After This Course, You Will Be Able To

- Design **multi-agent architectures**
- Choose the right **MAS design patterns**
- Implement agents using **Google ADK**
- Manage **agent communication & memory**
- Integrate **external tools and APIs**
- Build **modular agents with Skills**

Engineering Perspective

- When to use **Single Agent vs Multi-Agent**
- How to **orchestrate complex AI workflows**
- How to build **scalable AI systems**

2. From Single-Agent Systems to Multi-Agent

This section introduces the transition from single-agent systems to multi-agent architectures, highlighting their limitations, key concepts, architectural models, and the evolution of modern agentic AI systems.

Single-Agent Capabilities

What a Single-Agent System Can Do

- Process user input and generate responses
- Perform reasoning using LLMs
- Execute tools or APIs
- Maintain conversation context
- Handle simple task automation



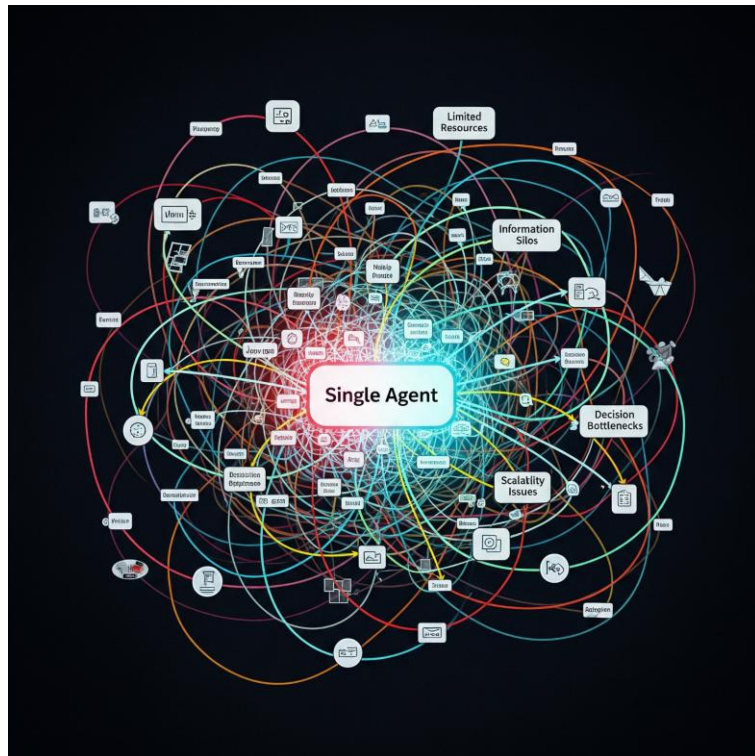
Limitations of Single-Agent Systems

Challenges in Complex Tasks

- Limited task specialization
- Increasing prompt complexity
- Difficult to scale large workflows
- Hard to maintain modular logic
- Reduced reliability in long reasoning chains

As Systems Grow

- Prompt becomes **too large**
- Context becomes **too noisy**
- Logic becomes **hard to maintain**



Why Multi-Agent Systems?

Motivation for MAS

- Break complex problems into smaller tasks
- Enable specialized agents
- Improve modularity and scalability
- Allow parallel and hierarchical workflows

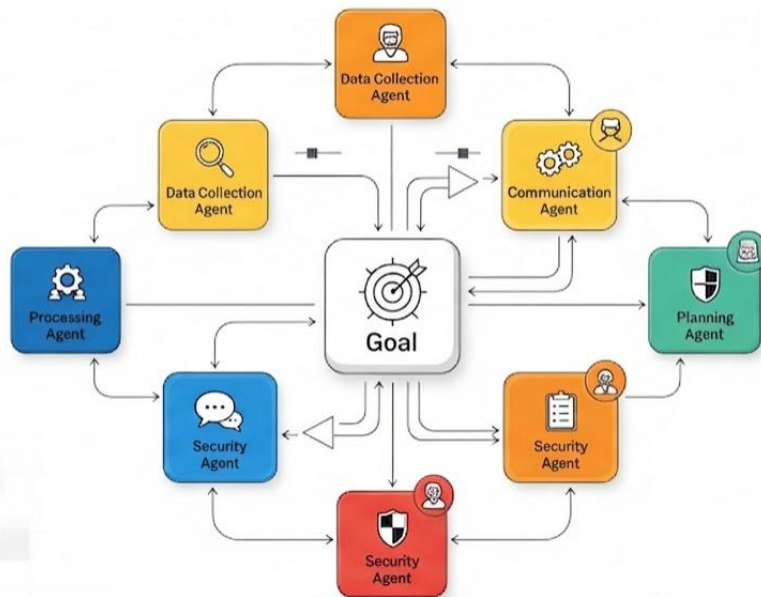
Key Benefits

- Task specialization
- Better reasoning structure
- Scalable architecture
- Improved system reliability



Example MAS Workflow

Multi-Agent System Workflow



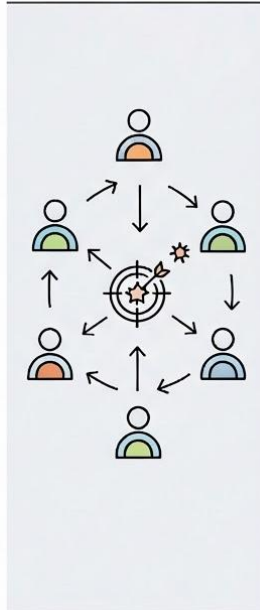
Key Principles of Multi-Agent Systems

Autonomy



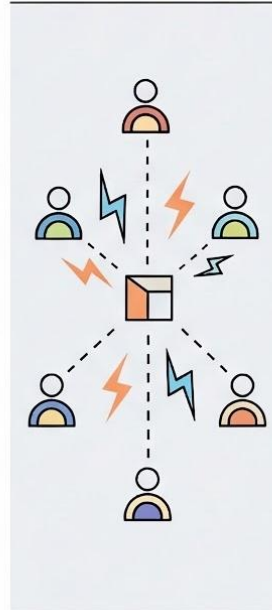
- Agents operate independently
- Each agent manages its own logic

Coordination



- Agents collaborate to achieve a shared goal
- Tasks are distributed across agents

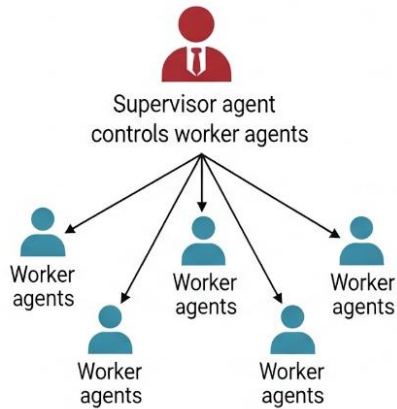
Communication



- Agents exchange information
- Messages, shared memory, or state

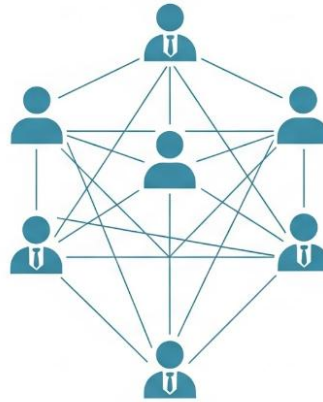
MAS Architecture Types

Hierarchical



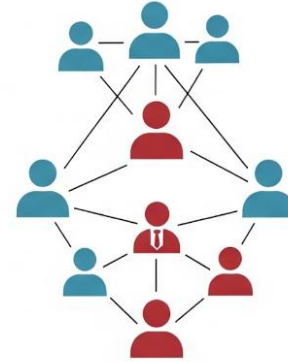
Clear task delegation

Decentraized



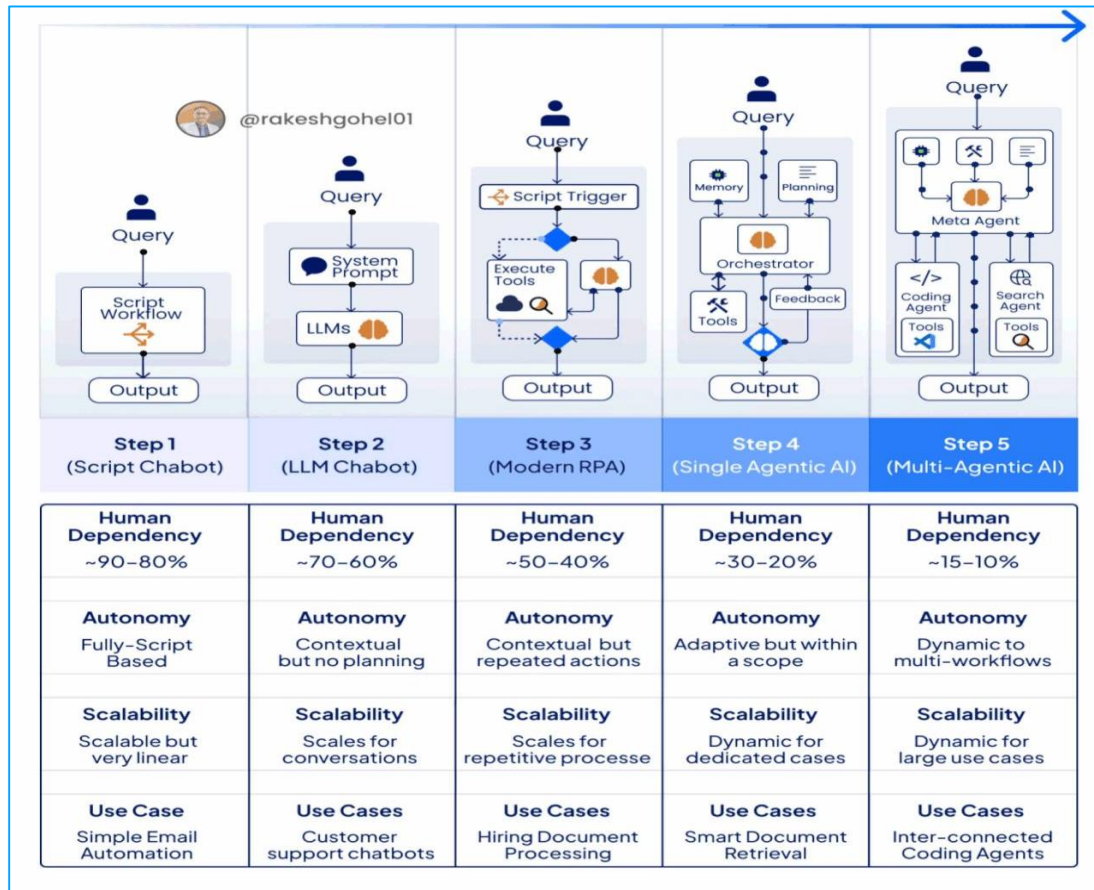
Agents interact without central controller
More flexible but harder to manage

Hybrid



Combination of centralized distributed control
Most common in production systems

Evolution of Agentic AI



When NOT to Use Multi-Agent Systems

Multi-Agent Systems Are Powerful – But Not Always Necessary

Use Single-Agent When:

- The task is simple and well-defined  Simple
- The workflow is short and linear  Linear
- No task specialization is required  Low complexity
- The system has low operational complexity

Examples:

-  Simple Q&A chatbot
-  Basic document summarization
-  Single API task automation

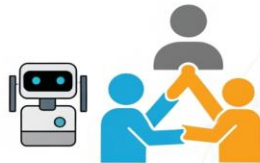
When Multi-Agent Systems Make Sense

Use Multi-Agent Systems When:

-  The problem requires multiple specialized skills
-  Tasks can be decomposed into smaller subtasks
-  The workflow is complex or multi-step
-  Agents must coordinate or collaborate

Examples

-  Research assistants
-  Enterprise copilots
-  Data analysis pipelines
-  Policy or compliance automation



Single-Agent vs Multi-Agent Decision Framework (1/2)

Criteria	Single Agent System (SAS)	Multi-Agent System (MAS)
Structure	There is only one AI Agent.	It consists of multiple AI agents that work together and interact with each other.
System complexity	Simple, minimal interaction.	It's more complicated because it requires coordination between different actors.
Coordination ability	No coordination is needed because there is only one agent.	Information sharing and collaboration are necessary to accomplish the task.
Scalability	Limited and difficult to expand due to dependence on a single agent.	Easily expandable by adding new agents to the system.
Fault tolerance	If a single agent fails, the entire system will shut down.	The system continues to function even if a few agents malfunction.

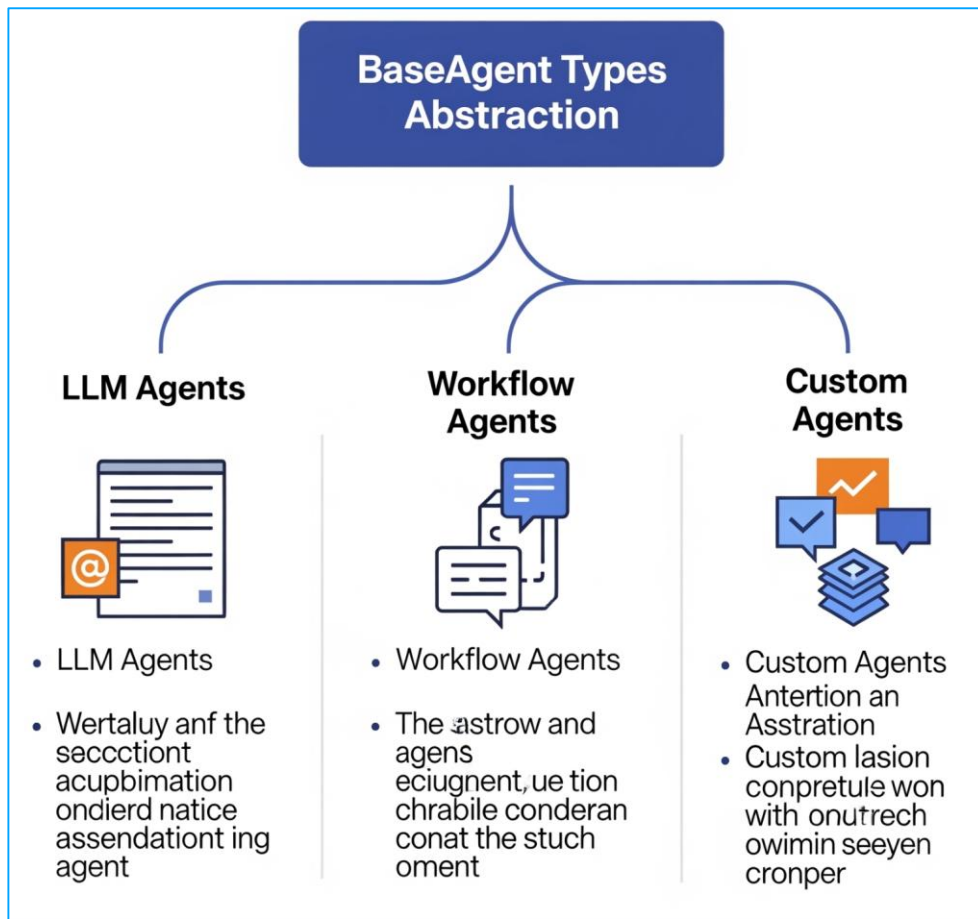
Single-Agent vs Multi-Agent Decision Framework (2/2)

Criteria	Single Agent System (SAS)	Multi-Agent System (MAS)
Decision-making process	The decision was made independently by an agent.	The agents make decisions together based on shared data.
Internal communication	No, because there is only one agent.	There is always communication and information exchange between agents.
Adaptability	Limited, dependent on the capabilities of a single agent.	High adaptability is achieved through collective learning and sharing of experiences.
Appropriate application	The task is simple and requires little interaction.	This is a complex, multi-dimensional problem that requires continuous collaboration.

3. Multi-Agent Architectures & Design Patterns with ADK

This section explores how multi-agent systems are designed using the Google Agent Development Kit (ADK). We examine core agent types, composition mechanisms, and common architectural patterns used to orchestrate collaboration between agents in complex AI workflows.

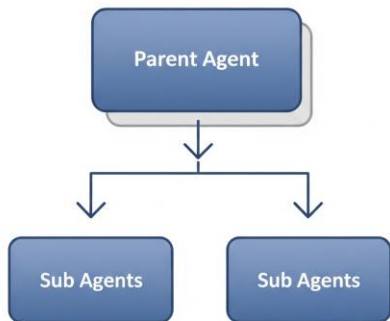
ADK Agent Types Overview



Agent Composition & Delegation in ADK

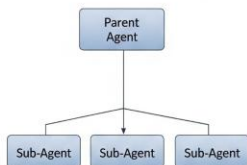
Building Multi-Agent Systems

Agents can be composited into hierarchies:
Parent Agent > Sub Agents

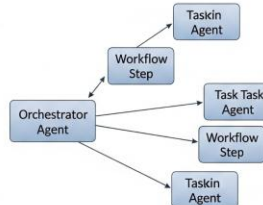


Agent Composition Primitives

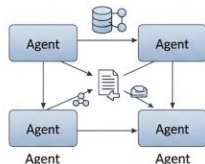
Agent hierarchy (parent / sub-agents)



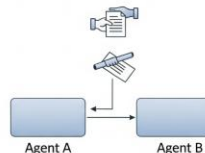
Workflow agents as orchestrators



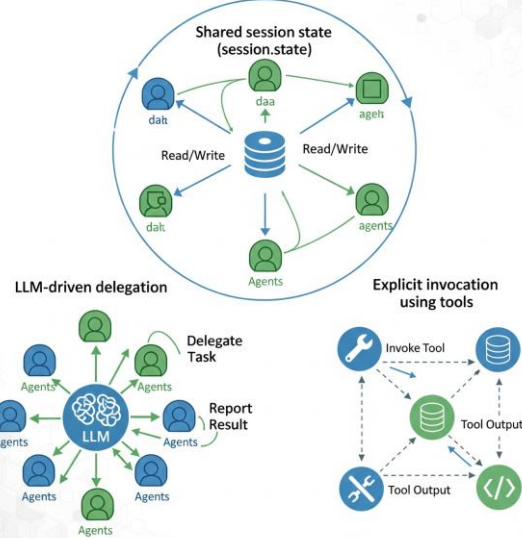
State sharing between agents



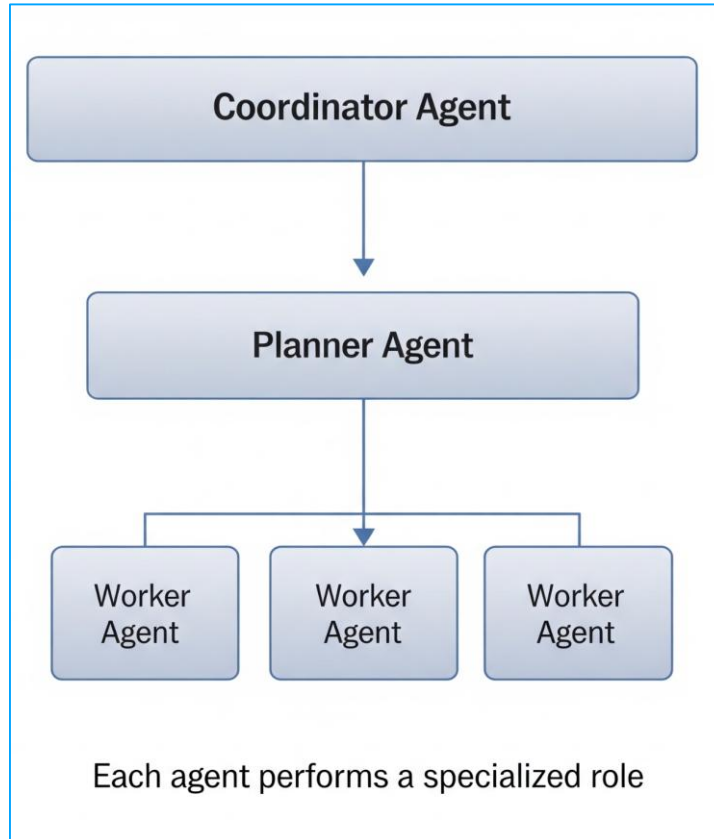
Delegation between agents



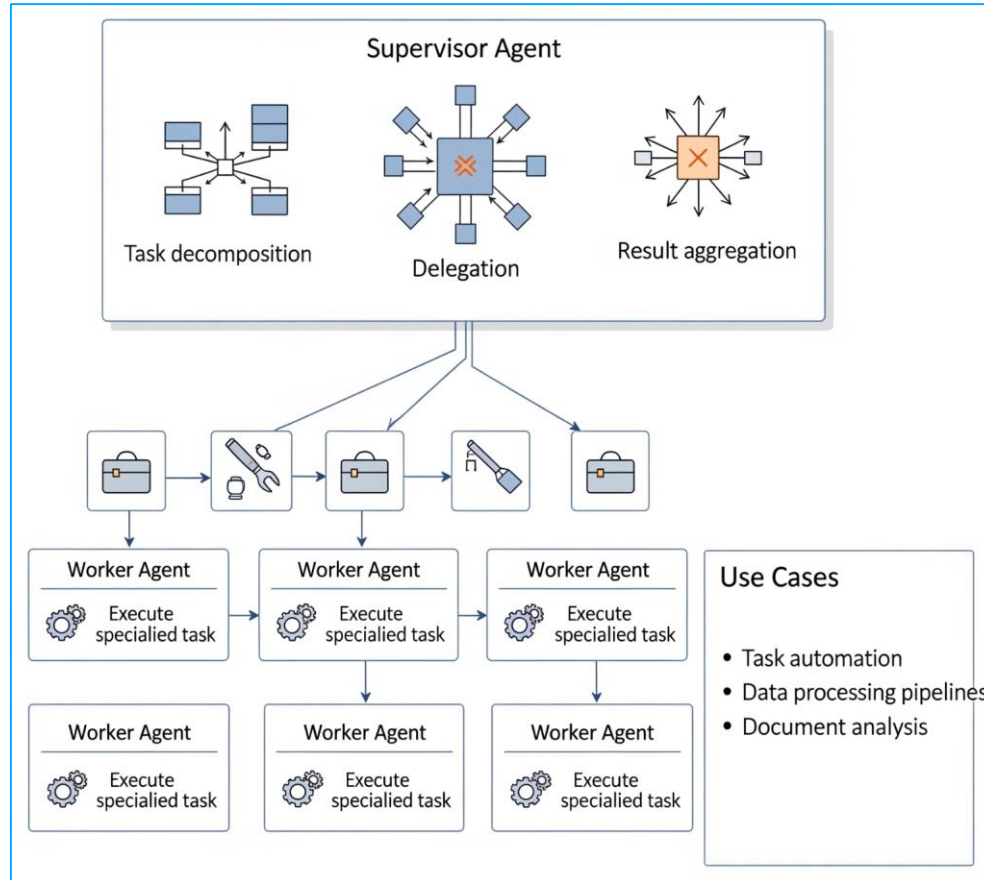
Communication Mechanisms



Example Architecture



Supervisor-Worker Pattern



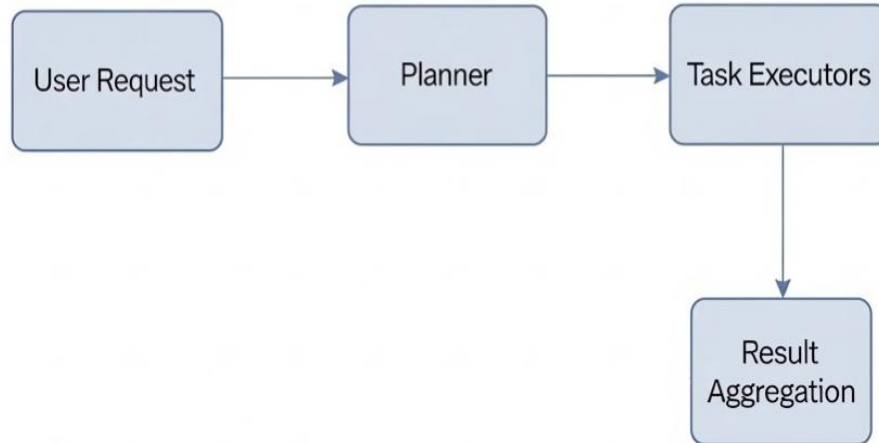
Planner–Executor Pattern

Planner Responsibilities

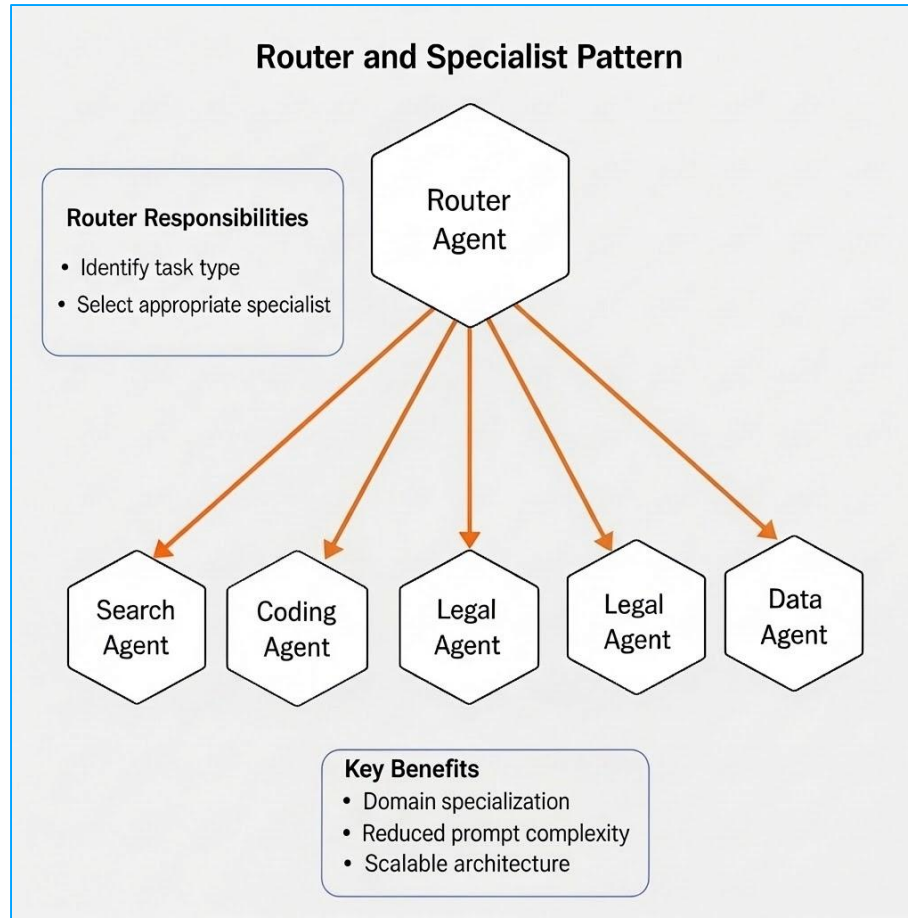
- Break complex tasks into subtasks
- Generate execution plan

Executor Responsibilities

- Perform individual tasks
- Return results

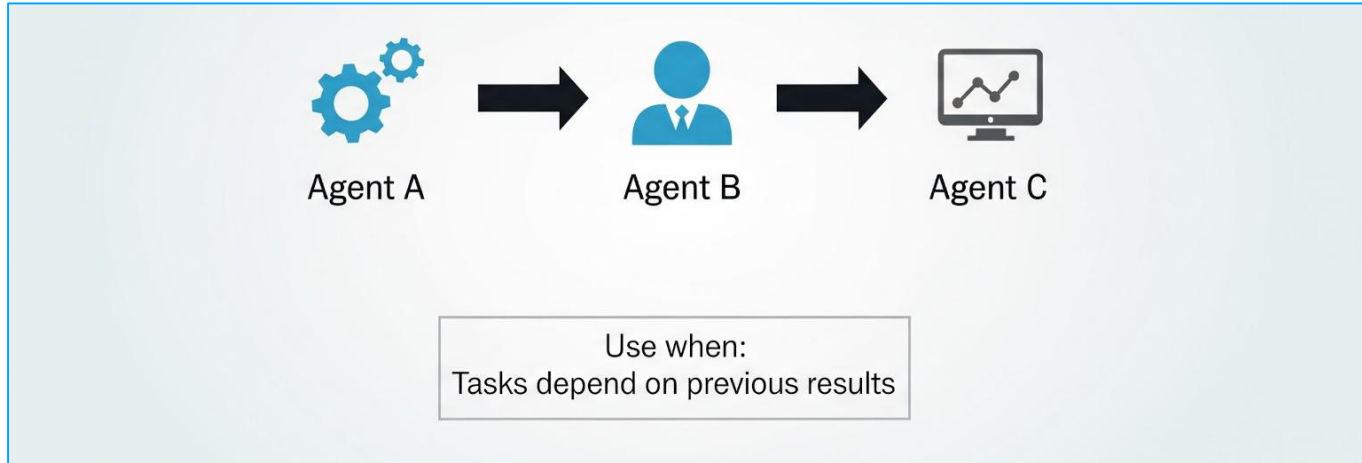


Router & Specialist Pattern



Workflow Patterns (1/3)

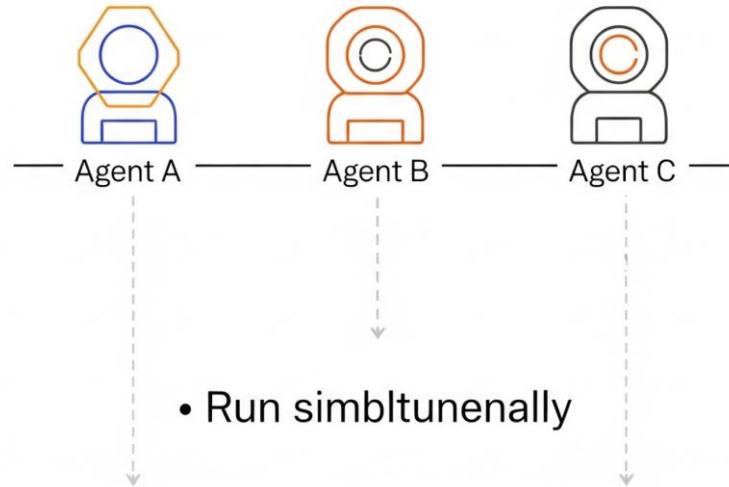
ADK provides built-in **workflow agents** for orchestration.



Sequential Pattern

Workflow Patterns (2/3)

Parallel Pattern



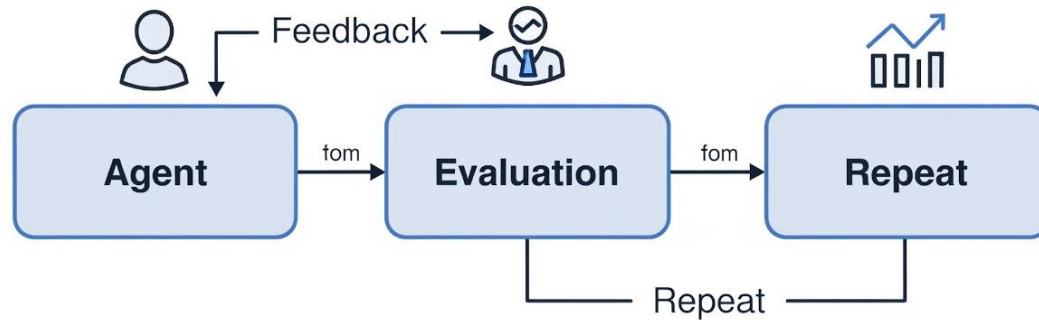
- Run simultaneously

☐ Use when: Tasks are independent

Workflow Patterns (3/3)

Loop Pattern

Use when: iterative refinement is required



Pattern Selection Strategy

How to Choose an Architecture

Consider:

- Task complexity
- Number of specialized skills
- Workflow dependencies
- Required reliability

Simple Task -> Single Agent

Moderate Complexity:

- Planner–Executor
- Router–Specialist

High Complexity:

- Supervisor–Worker
- Multi-agent workflows

4. Agent Communication & Memory Management in ADK

This section explains how agents maintain context and collaborate in ADK. We explore sessions, shared state, long-term memory, and observability mechanisms that enable reliable multi-agent coordination.

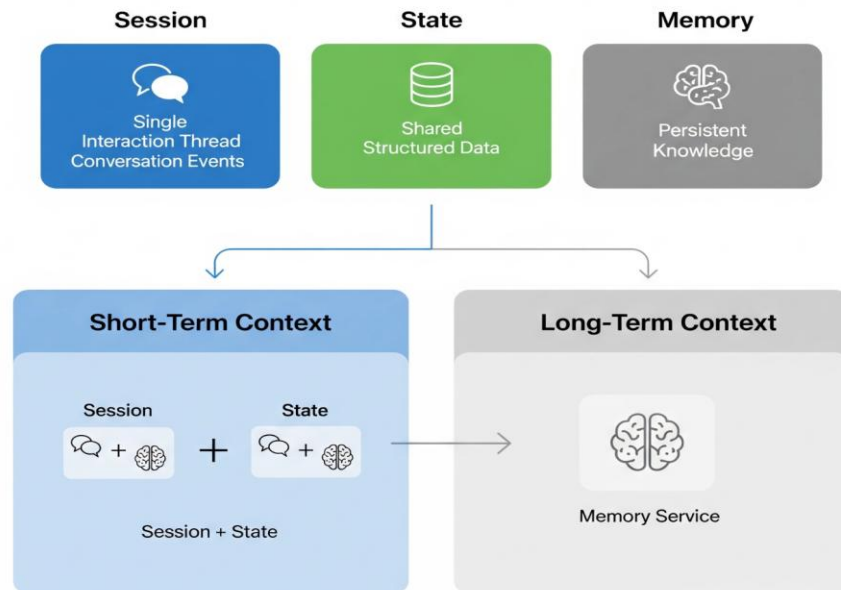
Sessions & Context Model

Why Context Matters

Agents need contextual information to:

- Maintain conversation continuity
- Track workflow progress
- Share information between agents

Core Context Components in ADK



Context Architecture



Session State Management

What is Session State?

`session.state`

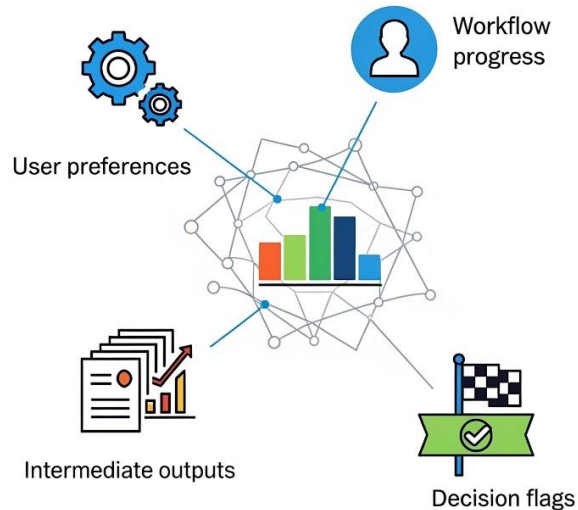
A shared **key-value store** used by agents during a session.

Example:

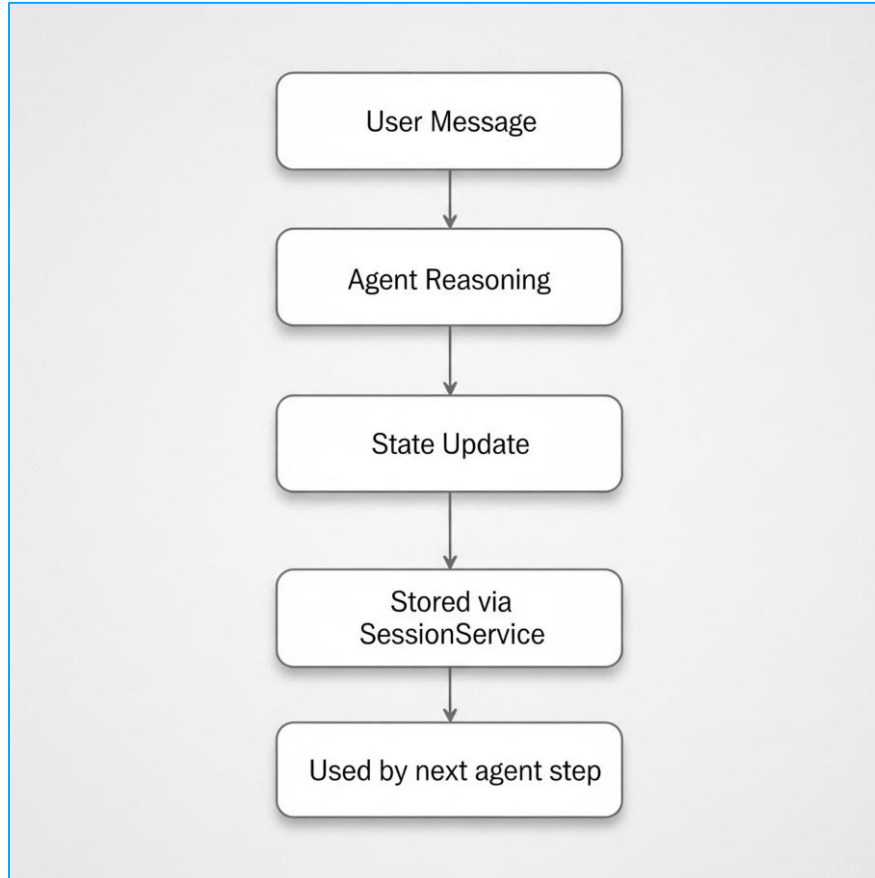
```
state["user_intent"] = "book_hotel"
```

```
state["workflow_step"] = "search_results"
```

Typical Data Stored



State Lifecycle



Inter-Agent Communication via State

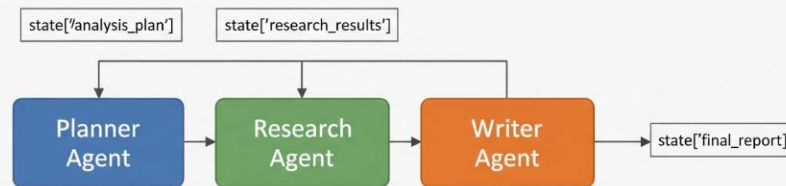
Communication Model in ADK

- Agents **do not directly message each other**.
- Instead they collaborate through: Shared Session State

Benefits

- Structured information exchange
- Persistent across workflow steps
- Enables agent orchestration

Multi-Agent



MAS Collaboration Model



Long-Term Memory with MemoryService

Why Long-Term Memory?

Session data is temporary.

Memory allows agents to:

- Learn from past interactions
- Store domain knowledge
- Retrieve information across sessions

Memory Service

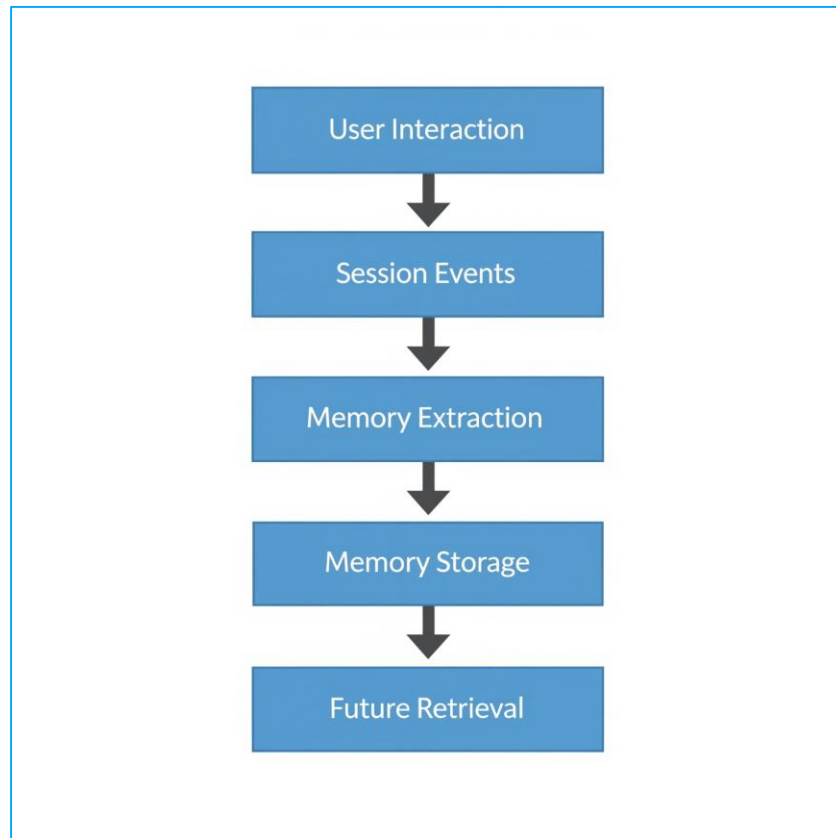
Core responsibilities:

- Store extracted knowledge
- Manage persistent memories
- Retrieve relevant information during reasoning

Typical Use Cases

- User preferences history
- Knowledge base retrieval
- Cross-session personalization

Memory Architecture



Memory Tools & Retrieval Strategies

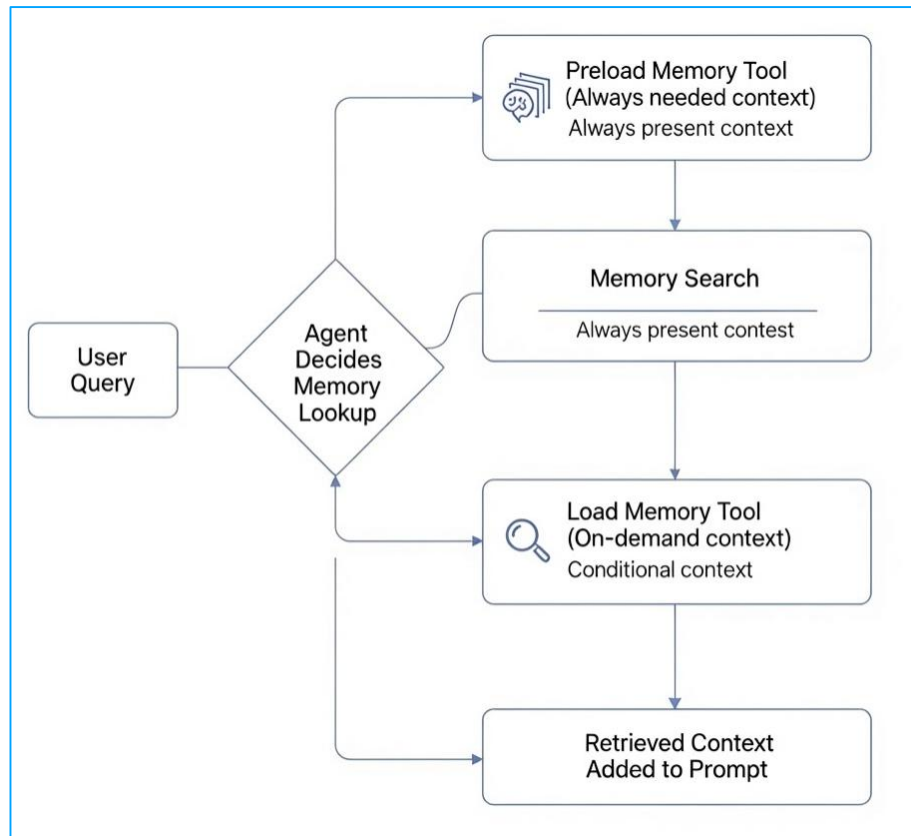
ADK provides tools for **memory retrieval**.

Preload Memory Tool

- Loads memory automatically at each turn
- Best for always-needed context

Load Memory Tool

- Triggered only when needed
- Reduces prompt size



Observability & Debugging

Multi-agent systems require **strong observability**.

Session Observability

- Track conversation history
- Inspect events and decisions

State Inspection

- Verify workflow progress
- Debug agent collaboration

Memory Debugging

- Monitor memory retrieval
- Validate context relevance

5. Integrating ADK Agents with Tools

This section introduces the tool integration model in Google Agent Development Kit (ADK). We explore how agents interact with external systems using function tools, APIs, MCP servers, and other extensible tooling mechanisms.

Tooling Architecture in ADK

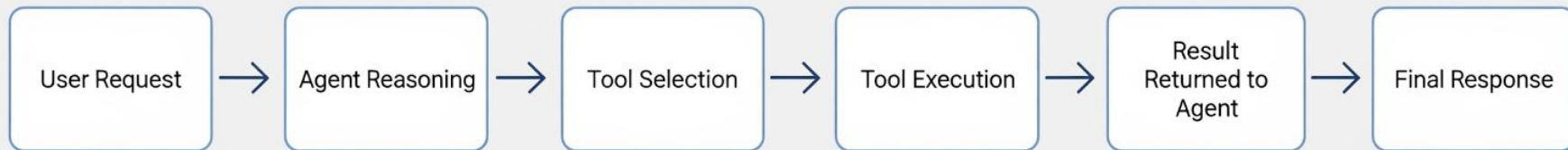
Why Agents Need Tools

LLMs alone cannot:

- Access real-time data
- Execute external actions
- Query databases or APIs

Tools enable agents to interact with the **external world**.

Agent + Tool Architecture



Custom & Function Tools

Function Tools

Simplest way to expose logic to an agent.

A **Python function becomes an agent tool.**

Example use cases:

- Database queries
- Internal services
- Utility operations

Custom Tools

Custom tools allow developers to build **advanced integrations.**

Typical scenarios:

- Enterprise system integrations
- Complex workflows
- Multi-step backend logic

Built-in Tools

ADK provides built-in tools for common tasks.

Examples include tools for:

- Memory retrieval
- File operations
- System utilities

Benefits

- Reduce development time
- Standardized agent capabilities
- Ready-to-use integrations

OpenAPI Tools

Purpose

Expose **REST APIs** as agent tools.

Agents can call external services using API specifications.

Example Use Cases

- CRM systems
- Payment APIs
- Weather services
- Internal microservices

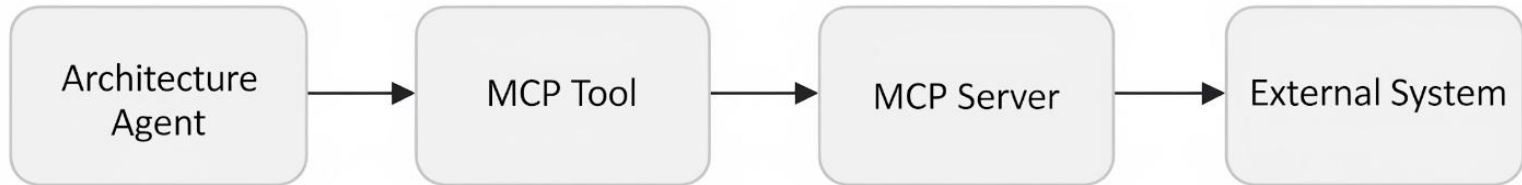


MCP Tools

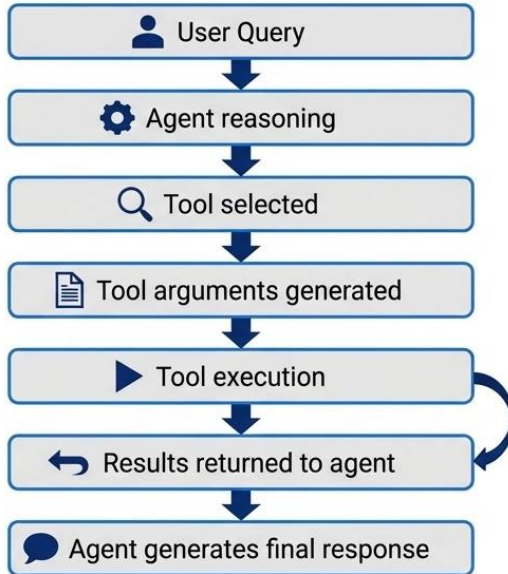
What is MCP?

MCP (Model Context Protocol) provides a **standard interface for tool servers**.

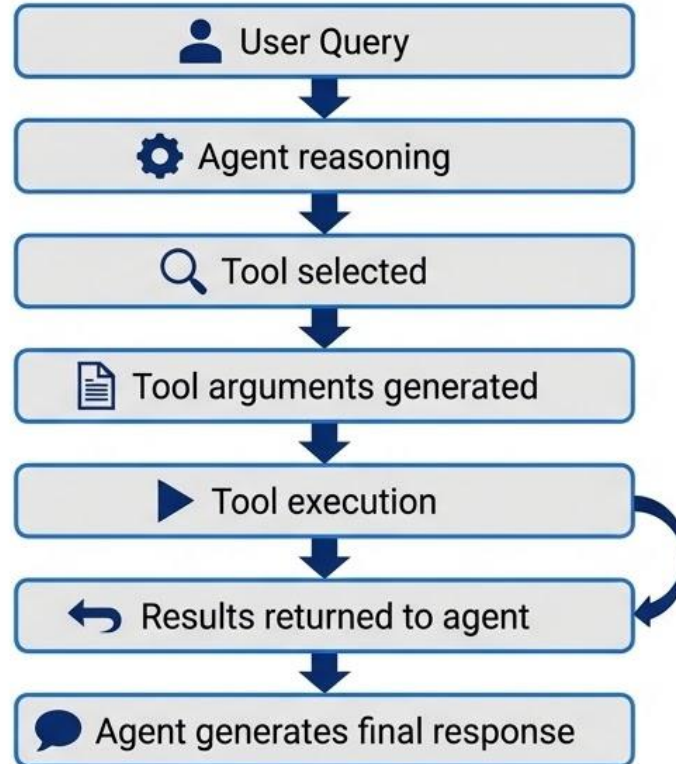
Agents connect to external systems via MCP servers.



Tool Execution Flow



Typical Tool Call Lifecycle



Tool Limitations & Best Practices

Common Limitations

- Latency
- External dependency failures
- API rate limits
- Security constraints

Best Practices

Design tools to be:

Simple -> Small focused functions.

Reliable -> Handle errors gracefully.

Secure -> Avoid exposing sensitive data.

Deterministic -> Return structured outputs when possible.

6. Skills for ADK Agents

This section introduces the **Skills framework in Google Agent Development Kit (ADK)**. We explore how skills package reusable capabilities, how they are structured, loaded, and executed, and when they should be used in real-world agent systems.

What Are Agent Skills?

Definition

Skills are reusable capability packages for agents.

They allow developers to bundle:

- Instructions
- Resources
- Knowledge
- Domain-specific behavior

into a reusable module.

Example Skills

- Customer Support Skill
- Travel Planning Skill
- Document Analysis Skill
- Compliance Checking Skill

Why Skills Matter

Skills help agents become:

Modular

Reusable across multiple agents.

Domain-aware

Specialized for specific tasks.

Maintainable

Easier to update and manage.

Skill Architecture

Metadata

Describes the skill.

Includes:

- name
- description
- tags

Helps the agent **discover and select the skill.**

Instructions

Guidelines for how the agent should perform the task.

Examples:

- reasoning strategy
- task workflow
- domain rules

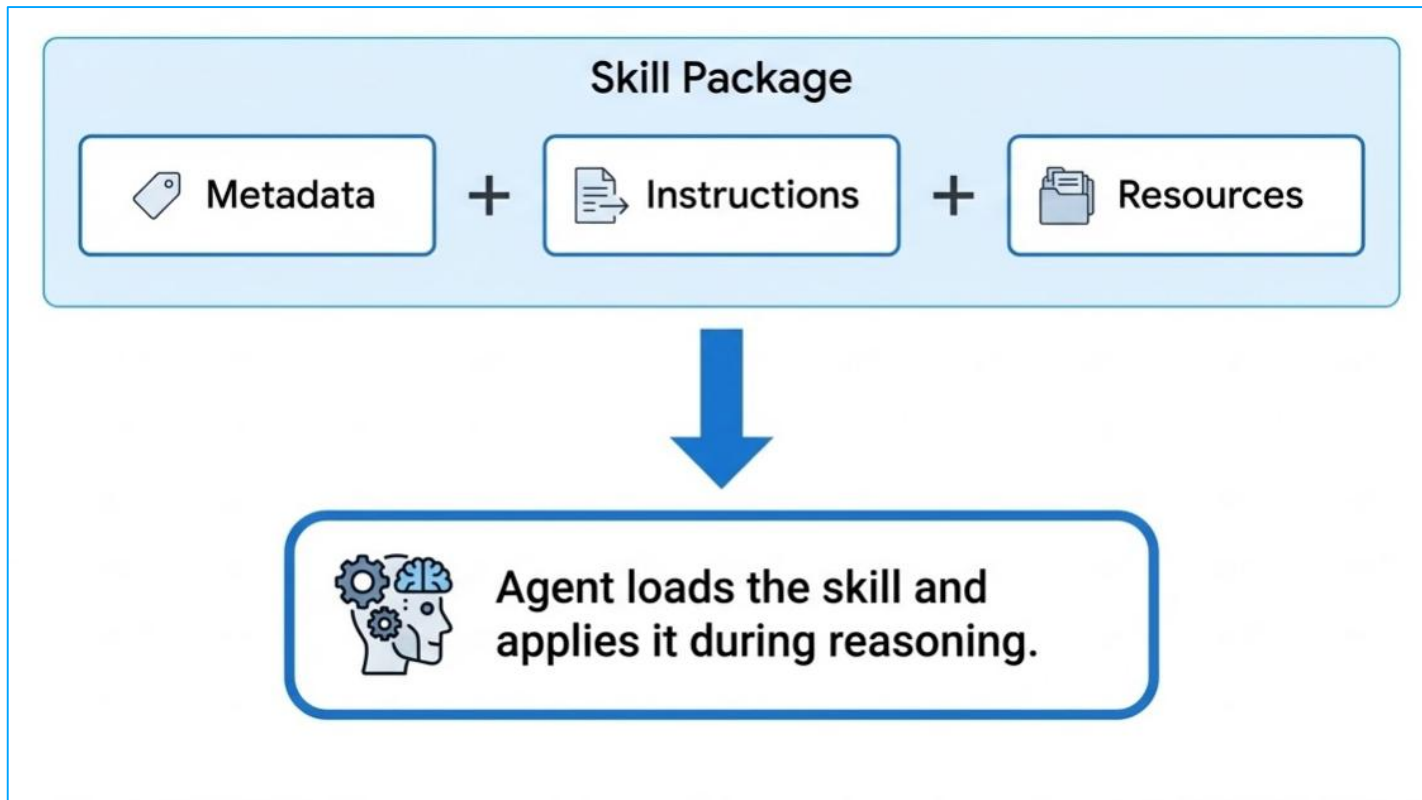
Resources

Supporting materials used by the skill.

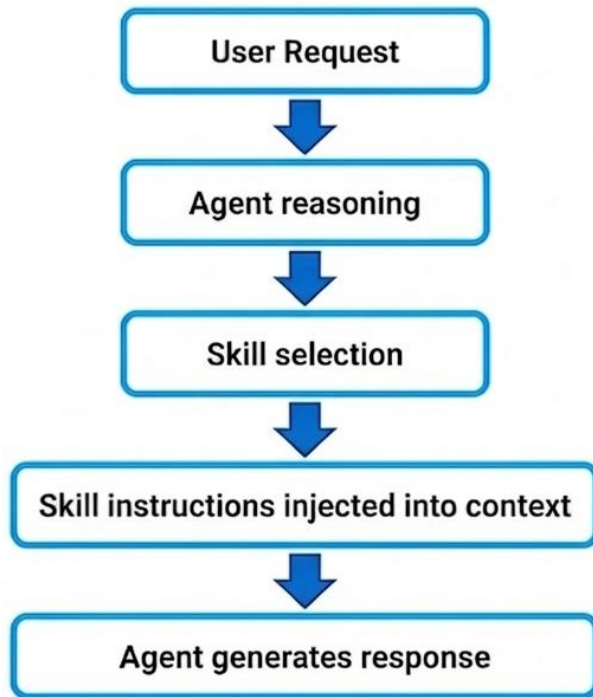
Examples:

- documents
- templates
- reference data
- prompt examples

Skill Architecture



Skill Execution Flow



File-Based vs Code-Defined Skills

ADK supports multiple ways to define skills.

File-Based Skills

Defined using files (e.g., YAML or structured configuration).

Benefits:

- Easy to modify
- Non-developers can contribute
- Quick iteration

Code-Defined Skills

Implemented directly in code.

Benefits:

- Full flexibility
- Dynamic logic
- integration with services

When to Use Skills

Skills are useful when agents need **domain expertise**.

Good Use Cases

Domain knowledge encapsulation

Examples:

- legal assistant
- finance advisor
- medical information agent

Reusable workflows

Examples:

- report generation
- document review
- structured analysis

Organizing complex prompt logic

Skills keep prompts structured and maintainable.

Limitations & Considerations

Skills Are Not Tools

- Skills guide reasoning but **do not execute actions**.
- External actions should use **tools**.

Prompt Size Constraints

- Large skills may increase prompt size.
- Keep instructions **focused and concise**.

Skill Selection Accuracy

- Agents must clearly understand when to apply a skill.
- Good **metadata descriptions are critical**.

7. Demo

8. Q&A and Technical Discussion